PROJECT SPOTLIGHT-WIN

Pixel-Perfect macOS Spotlight Launcher for Microsoft Windows 11 / 10

Document Type: Product Requirements (PRD) & Tech Spec **Architecture:** Tauri v2 + Rust Core + React 19 **Version:** 1.1.0 (Production & Auto-Updater Blueprint) **Date:** September 2026

1. Executive Summary & Product Vision

Project Spotlight-Win is an open-source, ultra-lightweight desktop launcher for Windows engineered to deliver an experience functionally and visually identical to Apple macOS Spotlight. While legacy utilities like PowerToys Run, Flow Launcher, and Wox offer file search capabilities, they often suffer from input latency, heavy background RAM usage, or distinct non-macOS visual aesthetics.

The primary objective of this project is to eliminate the design and performance gaps. By combining **Tauri v2 (Rust)** for near-zero runtime overhead with native **Windows DWM Acrylic/Mica** composition and the sub-millisecond **Voidtools Everything C-API SDK**, Spotlight-Win offers sub-10ms query execution while consuming less than 35MB of RAM at idle.

> **Core Value Proposition**
> Deliver sub-10ms response times, instantaneous global hotkey execution, 60 FPS glassmorphism animations, and native system file index integration inside a standalone 15MB executable with zero external runtime requirements.

2. Product Requirements Document (PRD)

2.1 Core Functional Requirements

Feature Module	Functional Description	Performance SLA	Priority
Global Hotkey Listener	Captures global `Alt+Space` or `Win+Space` low-level keyboard hooks across any active window without losing system focus state.	< 5 ms latency	P0 - CRITICAL
Frameless Glass Overlay	Centrally positioned floating input panel with native DWM Acrylic backdrop blur, 12px rounded corners, and dynamic height transition.	60 FPS / < 16ms frame	P0 - CRITICAL
Application Indexing	Parses Win32 Start Menu shortcuts (`.lnk`), UWP App Packages (`AppsFolder`), and system binaries present in system `%PATH%` .	Initial scan < 200ms	P0 - CRITICAL
Sub-10ms File Search	Direct integration with Voidtools Everything SDK C-API to query NTFS Master File Tables (MFT) across all attached drives.	< 8 ms query response	P0 - CRITICAL
Inline Evaluator	Instant parsing for arithmetic expressions, currency exchange rates, unit conversions, and world timezone comparisons.	< 1 ms calculation	P1 - HIGH
Dual-Pane Preview Card	Displays live preview for text files, images, PDFs, metadata, and web search summaries when navigating search results.	Render < 15ms	P1 - HIGH
System Quick Actions		Instant execution	P2 - MEDIUM

Feature Module	Functional Description	Performance SLA	Priority
	Executes power actions: Sleep, Lock, Shutdown, Restart, Empty Recycle Bin, Toggle Dark/Light Mode, and Volume controls.		

2.2 UX Micro-Interactions & Shortcut Specification

- **Invocation & Focus:** Pressing `Alt+Space` triggers an ease-out window slide-down animation (150ms). Focus immediately binds to the search input field.
- **Focus Loss Dismissal:** Listening to the Win32 `WM_KILLFOCUS` event instantly hides the window, resets the input buffer, and returns focus seamlessly to the previously active foreground window.
- **Keyboard Navigation:** Up `Arrow` / Down `Arrow` cycles selection through categorized search items. The dual-pane right-side preview updates synchronously without flickering.
- **Action Modifiers:**
  - `Enter` — Opens highlighted file or launches application.
  - `Ctrl+Enter` — Opens containing directory in Windows File Explorer with target file selected.
  - `Ctrl` (Hold) — Replaces file category subtitle with absolute destination file path.
  - `Spacebar` — Toggles full-screen Quick Look preview overlay for photos and documents.

3. Technology Stack & Framework Selection

Achieving the responsiveness of macOS Spotlight on Windows requires strict architectural constraints. Traditional web-based desktop engines like Electron carry a minimum memory floor of ~150MB to 250MB and introduce non-negligible input delay. Below is the framework selection matrix evaluated for this project:

Criteria	Tauri v2 + Rust (Selected)	C# / WinUI 3 (Alternative)	Electron + Node.js (Rejected)
Idle Memory (RAM)	**~28 - 35 MB**	~60 - 90 MB	~180 - 300 MB
Binary / Setup Size	**~12 - 15 MB**	~50 - 100 MB (.NET runtime)	~85 - 120 MB
Global Hotkey Latency	**< 2 ms (Direct Win32 API)**	< 3 ms (P/Invoke)	~15 - 30 ms (Node IPC)
UI Styling & Glass Blur	**React + Tailwind + DWM Glass**	XAML / Composition API	Chromium CSS (Emulated)
File Index Engine	**Rust C-FFI to Everything DLL**	C# P/Invoke to Everything DLL	Native C++ Node Addon

Final Tech Stack Architecture

- **Application Core & System Hooks:** Rust 1.80+ using `tauri-v2`, `tokio` (Async Task Executor), `windows-rs` (Win32 API bindings), and `global-hotkey` crate.
- **UI Layer & Rendering Engine:** React 19, TypeScript, Tailwind CSS, Framer Motion (60 FPS transitions), and Lucide React Icons. Embedded via Windows WebView2 runtime.
- **Window Dynamics & Aesthetics:** `window-vibrancy` crate providing direct access to Windows Desktop Window Manager (DWM) `SetWindowCompositionAttribute` for native Acrylic blur.
- **Search Engines:** Voidtools Everything SDK C-API (File Search), `sublime_fuzzy` / `nucleo` (Rust Fuzzy Scoring Engine), `evalexpr` (Math Evaluator).

4. System Architecture & Query Pipeline

The core architecture relies on an event-driven multi-threaded model where the frontend UI operates strictly as an asynchronous view layer, communicating with the underlying Rust core via high-speed IPC bridge calls.

4.1 Low-Latency Query Execution Pipeline

When the user enters text into the search bar, the input string is debounced by 5ms and dispatched to three concurrent Tokio worker threads:

1. **Worker A (Application & Shell Cache):** Queries in-memory trie containing all indexed Start Menu shortcuts, UWP apps, and Control Panel items using fuzzy scoring formulas:

$$Score = BaseFuzzyScore(Query, Target) + PrefixBonus - DistancePenalty$$

2. **Worker B (Inline Calculation & Unit Engine):** Parses input against math ASTs or currency conversion rates. Returns immediate result if input matches valid arithmetic patterns.
3. **Worker C (Everything SDK C-API):** Sends UTF-16 encoded string to the background Voidtools IPC service:

```
// Example Rust FFI binding to Voidtools Everything SDK
use std::ffi::OsStr;
use std::os::windows::ffi::OsStrExt;

#[link(name = "Everything64")]
extern "C" {
    fn Everything_SetSearchW(lpString: *const u16);
    fn Everything_QueryW(bWait: i32) -> i32;
    fn Everything_GetNumResults() -> u32;
    fn Everything_GetResultFilePathW(index: u32) -> *const u16;
}

pub unsafe fn query_everything_sdk(search_term: &str) -> Vec {
    let wide: Vec = OsStr::new(search_term).encode_wide().chain(Some(0)).collect();
    Everything_SetSearchW(wide.as_ptr());
    Everything_QueryW(1); // Wait for results

    let count = Everything_GetNumResults().min(50); // Top 50 results
    let mut results = Vec::new();
    for i in 0..count {
        let path_ptr = Everything_GetResultFilePathW(i);
        // Convert wide string pointer back to Rust String...
    }
    results
}
```

5. UI / UX Design & Pixel Fidelity Specification

5.1 Dimension & Typography Rules

- **Window Size:** Standard dimensions 750px width by 480px maximum height.

- **Border Radius:** 12px rounded outer shell with 1px solid border (`rgba(255, 255, 255, 0.15)` in dark mode).
- **Typography System:** Primary font family set to `Segoe UI Variable Text`, falling back to `-apple-system` / `system-ui`. Input font size set to 18pt with 0.2px letter spacing.
- **Color Palette (Dark Mode):**
  - Window Backdrop: `rgba(24, 24, 27, 0.75)` with 25px blur filter.
  - Active Selection Highlight: `#007aff` (macOS Accent Blue) or `rgba(255, 255, 255, 0.10)`.
  - Primary Text: `#f8fafc`; Secondary Text / Path: `#94a3b8`.

5.2 Rust Native Window Builder Setup

```
use tauri::{Manager, WebviewUrl, WebviewWindowBuilder};
use window_vibrancy::{apply_acrylic, apply_mica};

pub fn build_spotlight_window(app: &tauri::AppHandle) {
    let window = WebviewWindowBuilder::new(app, "main", WebviewUrl::App("index.html".into()))
        .title("Spotlight")
        .inner_size(750.0, 480.0)
        .resizable(false)
        .decorations(false)
        .transparent(true)
        .always_on_top(true)
        .center()
        .visible(false)
        .build()
        .expect("Failed to build spotlight window");

    #[cfg(target_os = "windows")]
    {
        // Apply native Windows DWM Acrylic translucent glass effect
        let _ = apply_acrylic(&window, Some((18, 18, 18, 125)));
    }
}
```

6. Packaging, Code Signing & GitHub Releases Distribution Pipeline

Deploying standalone Windows executables via GitHub Releases provides zero-cost hosting, reliable CDN performance, and an integrated auto-update mechanism.

6.1 Inno Setup Packaging Script (`installer.iss`)

```
[Setup]
AppId={{A8F3D129-9B21-4E82-B2A8-9842FDC1A910}
AppName=Spotlight Windows
AppVersion=1.1.0
DefaultDirName={autopf}\SpotlightWin
DefaultGroupName=Spotlight Windows
OutputDir=.\Output
OutputBaseFilename=SpotlightWin-Setup-v1.1.0
Compression=lzma2/ultra64
SolidCompression=yes
PrivilegesRequired=lowest

[Tasks]
Name: "autostart"; Description: "Start Spotlight automatically when Windows logs in"; Flags: unche

[Files]
Source: "..     arget
elease\spotlight-win.exe"; DestDir: "{app}"; Flags: ignoreversion
Source: "..     arget
elease\Everything64.dll"; DestDir: "{app}"; Flags: ignoreversion

[Icons]
Name: "{autoprograms}\Spotlight Windows"; Filename: "{app}\spotlight-win.exe"
Name: "{userstartup}\Spotlight Windows"; Filename: "{app}\spotlight-win.exe"; Tasks: autostart

[Run]
Filename: "{app}\spotlight-win.exe"; Description: "Launch Spotlight Windows"; Flags: nowait postin
```

6.2 GitHub Actions CI/CD Release Pipeline (`.github/workflows/release.yml`)

Automates building the application, signing updater assets, generating the Inno Setup installer, calculating SHA-256 checksums, and publishing assets alongside the `latest.json` manifest to GitHub Releases.

```yaml
name: Build & Release Spotlight Windows

on:
  push:
    tags:
      - 'v*'

jobs:
  build-windows:
    runs-on: windows-latest
    steps:
      - uses: actions/checkout@v4

      - name: Setup Node.js & Rust
        uses: dtolnay/rust-toolchain@stable

      - name: Install Frontend Dependencies
        run: npm ci

      - name: Build & Package Tauri App with Updater Manifest
        uses: tauri-apps/tauri-action@v0
        env:
          GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
          TAURI_SIGNING_PRIVATE_KEY: ${{ secrets.TAURI_PRIVATE_KEY }}
        with:
          includeUpdaterJson: true
          tagName: ${{ github.ref_name }}
          releaseName: 'Spotlight Windows ${{ github.ref_name }}'

      - name: Generate Inno Setup Installer
        uses: LagDaemon/InnoSetup-Action@v1
        with:
          filepath: 'installer.iss'

      - name: Calculate SHA-256 Checksum
        run: |
          CertUtil -hashfile Output/SpotlightWin-Setup-v1.1.0.exe SHA256 > Output/checksum.txt

      - name: Attach Installer to GitHub Release
        uses: softprops/action-gh-release@v1
        with:
          files: |
            Output/SpotlightWin-Setup-v1.1.0.exe
            Output/checksum.txt
        env:
          GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
```

6.3 Native In-App Updater Integration

Spotlight-Win checks GitHub Releases automatically on boot by querying the static `latest.json` file generated during CI/CD builds.

1. Tauri Updater Endpoint Configuration (`tauri.conf.json`)

```
{
  "plugins": {
    "updater": {
      "pubkey": "dW50cnVzdGVkIGNvbW1lbnQ6IGF1dG8tdXBkYXRlIHB1YmxpYyBrZXk...",
      "endpoints": [
        "https://github.com/YOUR_USERNAME/spotlight-win/releases/latest/download/latest.json"
      ]
    }
  }
}
```

2. Client-Side Auto-Update Verification (React / TS)

```
import { check } from '@tauri-apps/plugin-updater';
import { relaunch } from '@tauri-apps/plugin-process';

export async fn checkAndApplyUpdates() {
    try {
        const update = await check();
        if (update?.available) {
            console.log(`New version found: ${update.version}`);
            await update.downloadAndInstall();
            await relaunch(); // Restart application with updated binary
        }
    } catch (error) {
        console.error('Failed to check for updates:', error);
    }
}
```

7. Implementation Roadmap & Milestones

Phase	Target Deliverables	Estimated Duration
Phase 1: Shell & Hotkey	Setup Tauri v2 boilerplate, register Win32 `Alt+Space` global hotkey, apply Acrylic blur window effect, handle `WM_KILLFOCUS` auto-hide logic.	Week 1
Phase 2: Core Search Engine	Build Start Menu shortcut scanner, integrate Everything SDK C-API DLL bindings, implement in-memory application cache and fuzzy matching algorithm.	Week 2
Phase 3: UI & Dual-Pane Preview	Implement React 19 visual UI layer, pixel-perfect macOS typography, math evaluation cards, image/file right-pane preview cards, and keyboard navigation.	Week 3
Phase 4: Distribution & Auto-Updater	Configure Inno Setup installer, setup Tauri updater plugin with `latest.json` , build GitHub Actions pipeline, and publish initial release.	Week 4